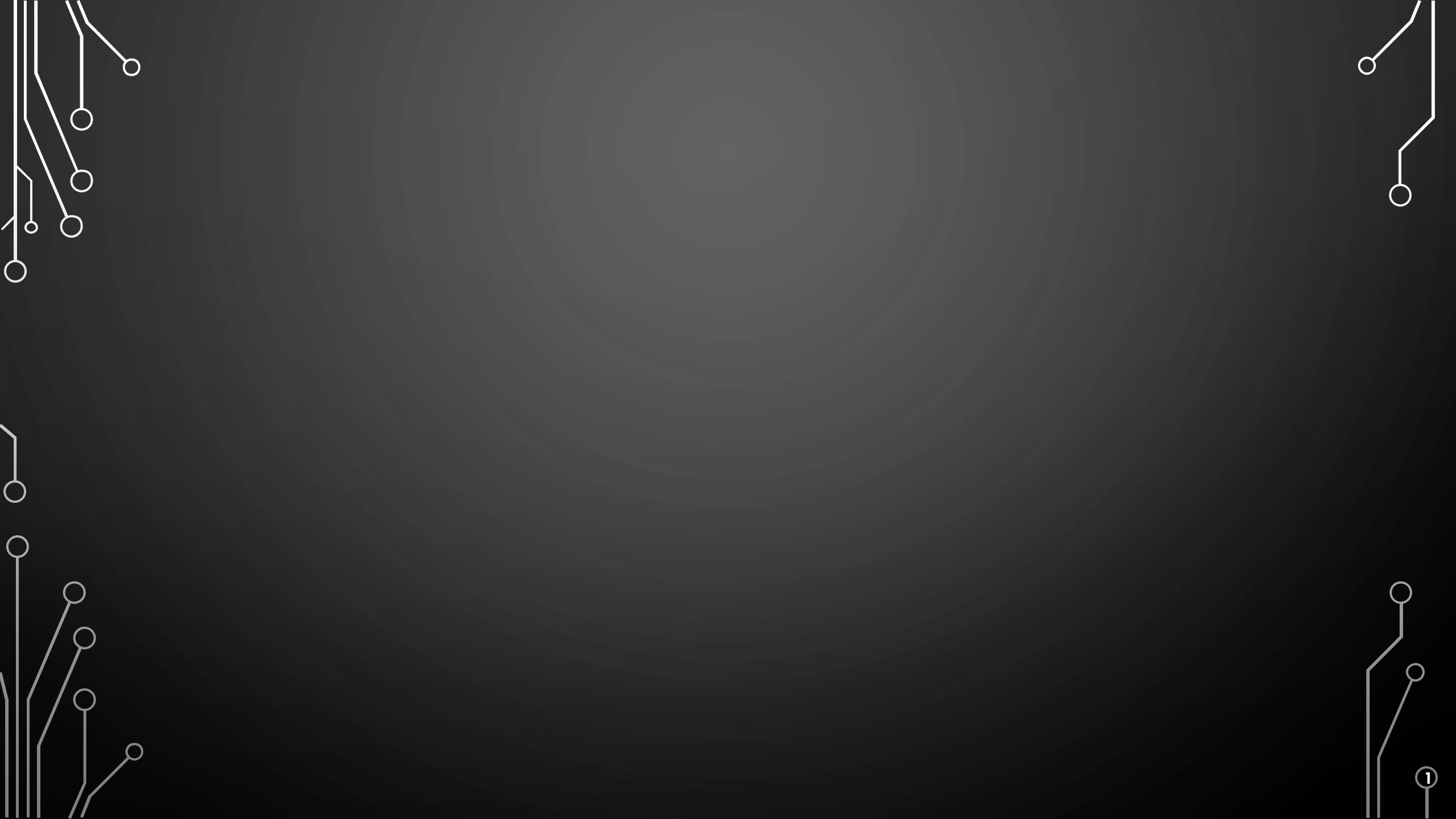
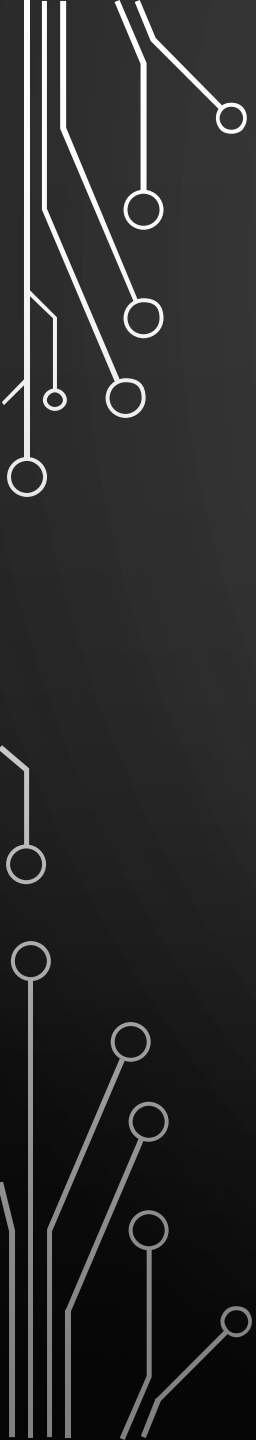




The background features a dark gray gradient with white circuit-like lines and nodes. These lines are primarily located along the left and right edges, with some extending towards the center. The nodes are small white circles at the end of the lines.

BLACKBOX TESTING: **EQUIVALENCE** **PARTITIONING and** **BOUNDARY VALUE** **ANALYSIS**

BLACKBOX TESTING: INTRODUCTION

BB testing: “a procedure to derive and/or select test cases based on an analysis of the specification, either functional or non-functional, of a component or system without reference to its internal structure.”

A large black downward-pointing triangle with a white border. Inside the triangle, the text 'BlackBox testing techniques' is written in white, bold, sans-serif font.

BlackBox testing techniques

Techniques and topics:

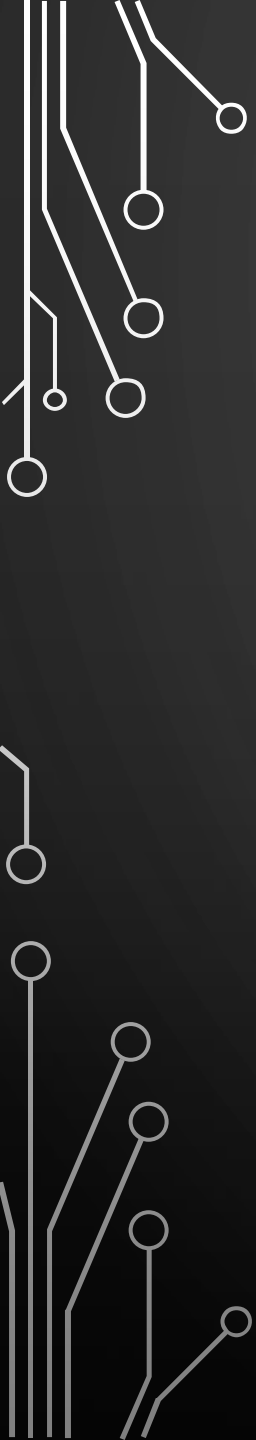
Decision table testing

State transition testing

Equivalence partitioning

Boundary value analysis

Use case testing

The background features a dark gray gradient with faint, stylized circuit board traces in white and light gray. These traces are primarily located along the left and right edges of the slide, with some branching patterns extending towards the center. The main title is centered in the upper half of the slide.

BLACKBOX TESTING: **EQUIVALENCE PARTITIONING**

BLACKBOX TESTING: EQUIVALENCE PARTITIONING

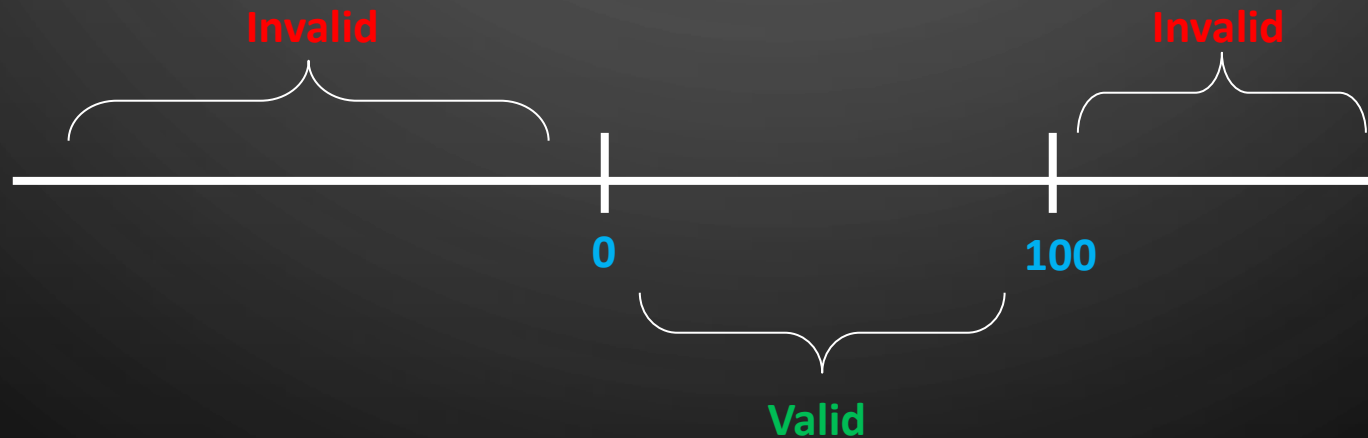
- What is **Equivalence Partitioning** for?
 - Designed to minimize number of test cases while still covering a large amount system requirements.
- How does **Equivalence Partitioning** work?
 - All test inputs are divided into groups called **Equivalence Partitions**.
 - Test inputs include human inputs, input via interfaces to the system, interface parameters in integration testing, and inputs from hardware, etc.
 - Each **Equivalence Partition** should be processed by the system in a similar way, signifying expectation of equivalent results.
 - Testing a single member of an **Equivalence Partition** should be equivalent to testing any other partition member.

BLACKBOX TESTING: EQUIVALENCE PARTITIONING

For example 1 a): Let's consider we have a function that is meant to take in inputs as positive Integer between 0 and 100. We may consider the following 3

Equivalence Partitions:

- **Valid** (+90, +10...)
- **Invalid** (+110, -10...)

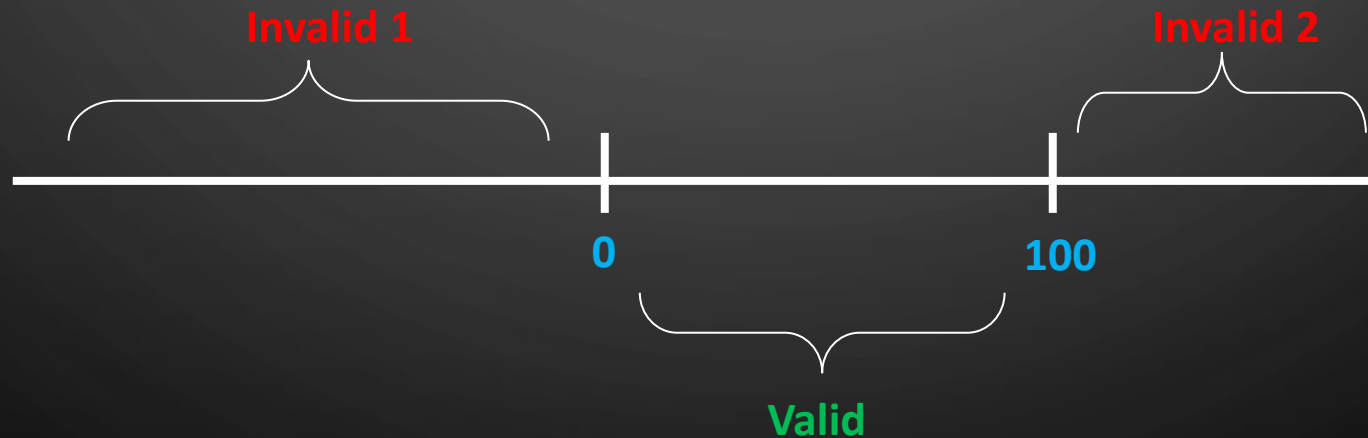


BLACKBOX TESTING: EQUIVALENCE PARTITIONING

For example 1 b): Let's consider we have a function that is meant to take in inputs as positive Integer between 0 and 100. We may consider the following 3

Equivalence Partitions:

- Valid (+90, +10...)
- Invalid 2 (+110...)
- Invalid 1 (-10...)



BLACKBOX TESTING: EQUIVALENCE PARTITIONING

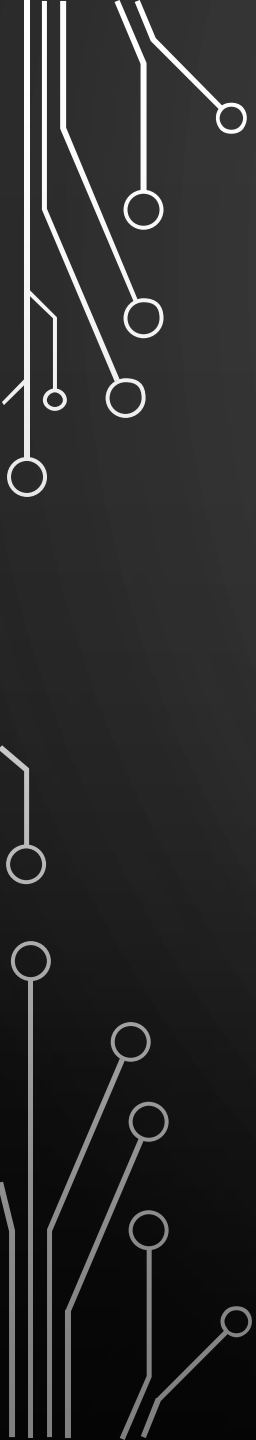
For example 2: Let's consider binary search in an unsorted array of 10 entries of positive integers lower than 99, the **Equivalence Partitions** could be:

- Inputs where precondition==**true** and element **is** an array member, e.g. 12.
- Inputs where precondition==**true** and element **isn't** an array member, e.g. 95.
- Inputs where the precondition==**false**, e.g. -19, 150, 1000, etc.

45	10	7	8	12	56	17	11	73	50
----	----	---	---	----	----	----	----	----	----

BLACKBOX TESTING: EQUIVALENCE PARTITIONING

Disadvantages	Advantages
• Excludes some inputs • No rule to choose inputs • Very limited focus • No guarantee that all elements of an Equivalence Partition are treated in the same way	• Usually easy to analyze • Applicable for all test levels • Minimizes amount of test cases while still covering a large amount system requirements • Can be used to partition both inputs and outputs

The background features a dark gray gradient with faint, stylized circuit board traces in white and light gray. These traces are primarily located along the left and right edges of the frame, with some extending towards the center. The traces consist of straight lines and small circles, resembling electronic components or wiring.

BLACKBOX TESTING: **BOUNDARY VALUE** **ANALYSIS**

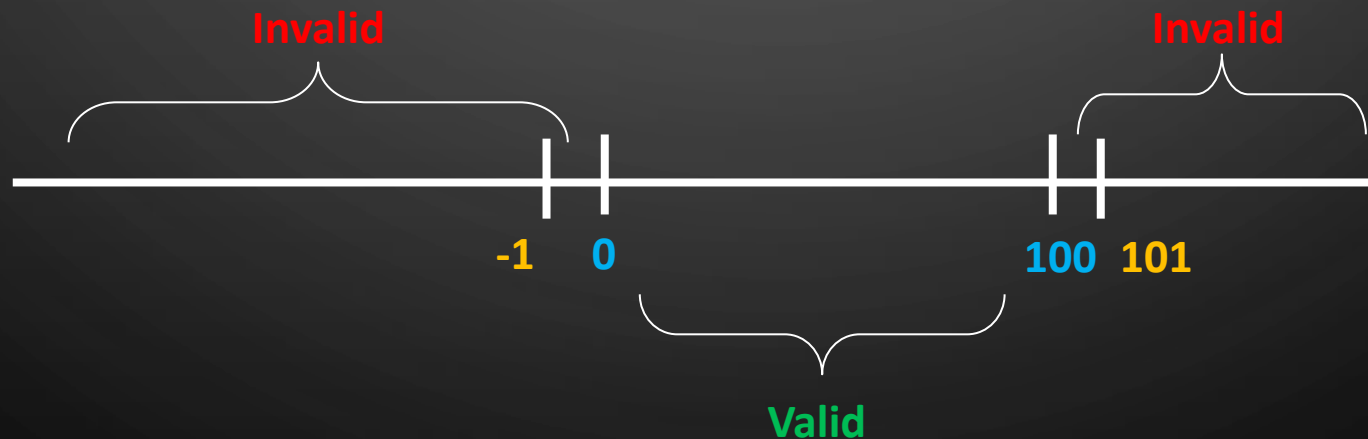
BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

- What is **Boundary Value Analysis** for?
 - Designing test cases based on boundary value inputs and to ensure proper functionality of the system. Assumes testing boundaries is likely to yield defects
 - Complement to **Equivalence Partitioning** by selecting the max and min values of partitions for boundary values.
- How does **Boundary Value Analysis** work?
 - Test input values are selected at, and just beyond the boundaries of **Equivalence Partitions**—choosing values on each boundary.
 - Tests cover both **valid** and **invalid** boundaries.
 - Both input and output boundaries are used to determine tests.

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

For example 1 [as seen for **Equivalence Partitioning** example 1 a)]: Let's consider we have a function that is meant to take in inputs as positive Integer between 0 and 100. We may consider the following **boundary values**:

- **Valid**: 0, 100 (low & high) (in some interpretations, we could add 1, 99, to test each boundary with 3 Tests).
- **Invalid**: -1, 101 (low & high).



BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

As well as value, we need to consider both **input representation and format**.

- Continuing the previous example (1):
 - Now assume that inputs must be in a 3 digit format (system will complete entry with leading 0s).

Invalid: if you try to enter a null value, the system won't allow it.

Valid: Try to enter 1 digit and verify system (system adds 2 leading zeros).

Valid: Try to enter 3 digits.

Invalid: If you try to enter 4 digits the system won't allow it.

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

- For example 2 (as seen for **Equivalence Partitioning**): Let's consider binary search in an unsorted array of 10 entries of positive integers lower than 99, the **Boundary Values** could be:
 - Inputs where precondition==**true**, e.g. 0, 98.
 - Inputs where precondition==**true** and element **is** the 1st array member, 45.
 - Inputs where precondition==**true** and element **is** the last array member, 50.
 - Inputs where precondition==**true** and element **isn't** an array member, e.g. 95.
 - Inputs where the precondition==**false**, e.g. -1, 99.

45	10	7	8	12	56	17	11	73	50
----	----	---	---	----	----	----	----	----	----

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Considering **String** inputs: length and collating sequence (order in which characters are sorted—e.g., ASCII)

- General rule, test for: empty string, one char string, typical string, and max size string.

Let's consider **boundary values** for example 3: enter "First Name", between **1 – 10** CAPITAL letter characters.

Invalid: First Name:

- Null,
- Lower invalid character: "@" (ASCII 64)

First Name:

• **Valid:** First Name of

- 10 characters "BORDERNAME"
- Upper Valid: "Z"

• **Valid:** First Name of

- one character: "G"
- Lower Valid: "A"

(Note: if we combined these into one test, we'd be unsure whether it fails due to lower boundary of ASCII or lower boundary of string size.)

Invalid: First Name of

- 11 characters, "FAKEBIGNAME"
- Upper Invalid: "[" (ASCII 91)

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Considering **Date** inputs:

- format (dd/mm/yy, mm/dd/yyyy, etc.),
- restrictions within the calendar (# of days in a month)
 - don't forget leap years.
- Let's consider **boundary values** for example 4: Enter "start date" in dd/mm/yyyy format.
 - **Assumptions:**
 - None of them are older than 120 years.
 - Date can be in the future, but **not beyond** today + 1 Month (31 days).

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Param	Valid	Invalid
dd	• Valid low boundary: 1 • Valid high boundary: #of_days_in_month (28, 29, 30 or 31)	• Invalid low boundary: 0 • Invalid high boundary: #of_days_in_month + 1 (29 or 30 or 31 or 32)
mm	• Valid low boundary: 1 • Valid high boundary: 12	• Invalid low boundary: 0 • Invalid high boundary: 13
yyyy	• Valid low boundary: today - 120 • Valid high boundary: Today + 31 days.	• Invalid low boundary: Today - 121 • Invalid high boundary: Today + 32 days

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Considering **Time** inputs:

- Format (HH:MM; HH:MM:SS),
- 12 or 24 hour clock.

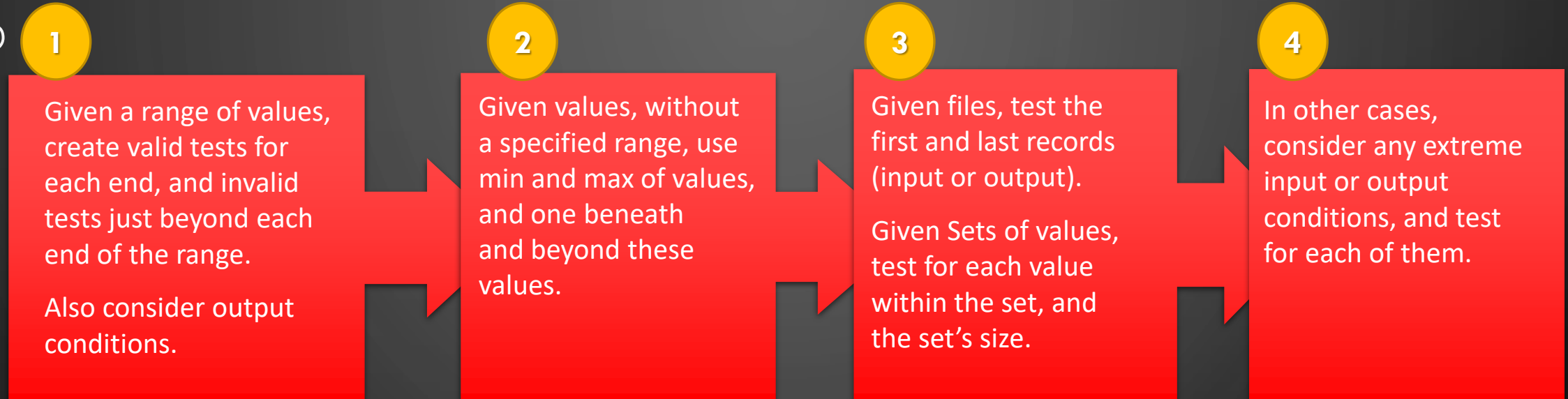
Let's consider **boundary values** for example 5: enter "start time" in format HH:MM.

- **Assumptions:** 24 hours clock, and **Time** can be in the future.

Parameter	Valid	Invalid
hh	• Valid low boundary: 00 • Valid high boundary: 23	• Invalid low boundary: -01 • Invalid high boundary: 24
mm	• Valid low boundary: 00 • Valid high boundary: 59	• Invalid low boundary: -01 • Invalid high boundary: 60

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Consider a general process for generating boundary tests:



Example output conditions: Billing system produces a report, in which total number of subscribers mustn't exceed 6,000,000.

Valid outputs: 0 and 6,000,000

Invalid outputs: -1 and 6,000,001

BLACKBOX TESTING: BOUNDARY VALUE ANALYSIS

Disadvantages	Advantages
• Excludes some inputs. • Doesn't consider dependencies between the combinations of input values.	• Applicable for all test levels. • Can expose potential user interface/user input problems. • Very clear guidelines for determining tests. • Capable of identifying a high amount of defects.



Copyright © 2023 by Ryan Fraser Kirwan

All rights reserved. No part of this video may be reproduced, or transmitted in any form or by any means, including recording, or other electronic or mechanical methods, without the prior permission of the creator, except in the case of certain other noncommercial uses permitted by copyright law.